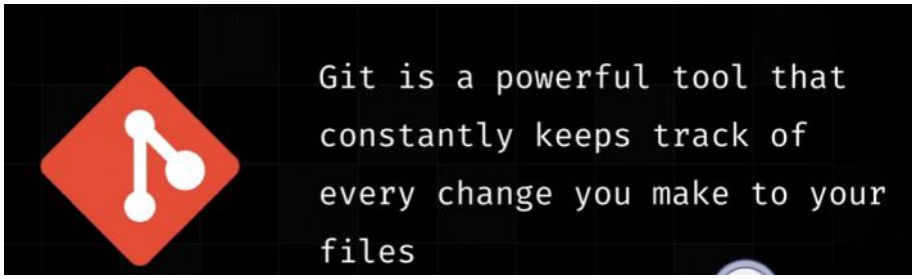
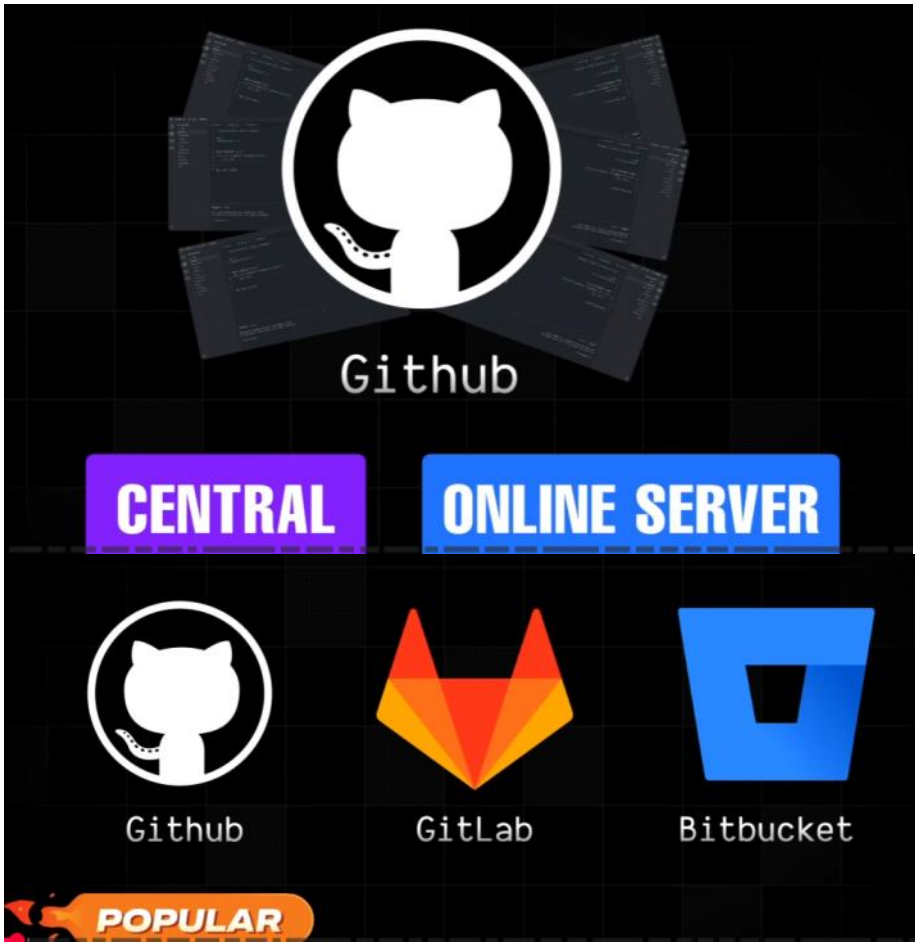
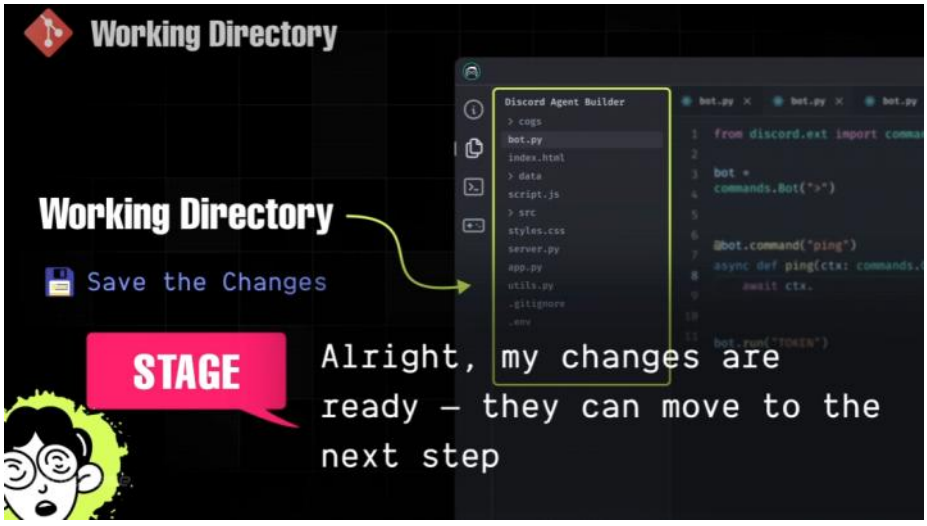
Git

10 May 2026 11:24 PM







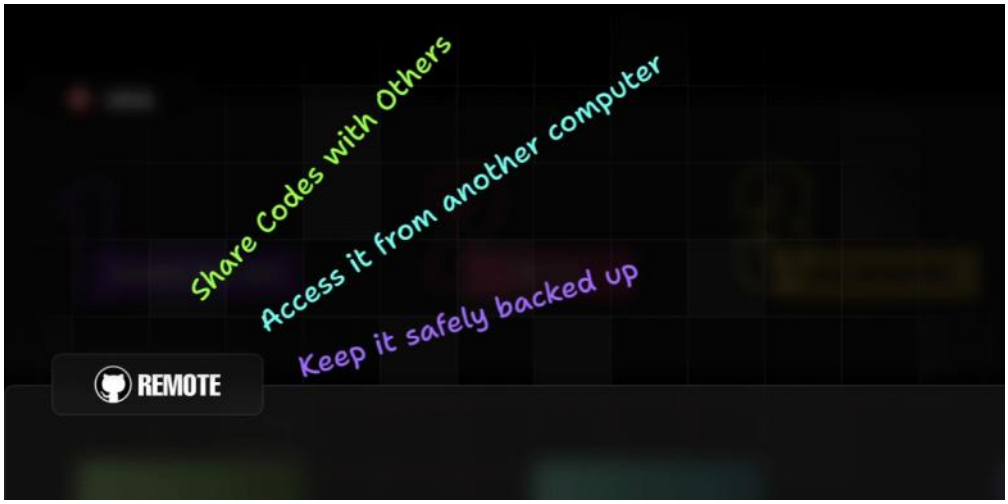
1
WORKING DIRECTORY
The folder where you're working on your project
2
STAGE
My changes are ready they can move to the next step
3
Local Git's Workflow
SUBSCRIBE

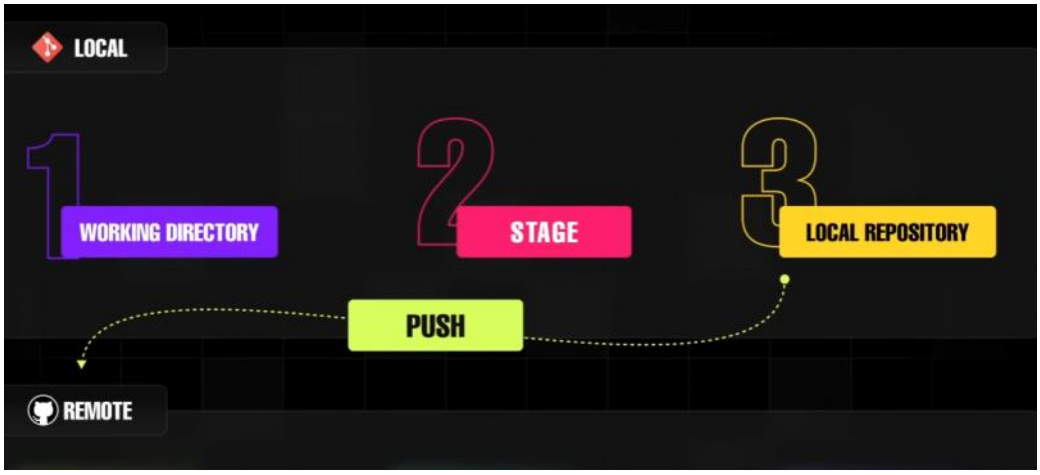
3
LOCAL REPOSITORY
Temporary area where files sit between working directory and repository save.
Local Git's Workflow

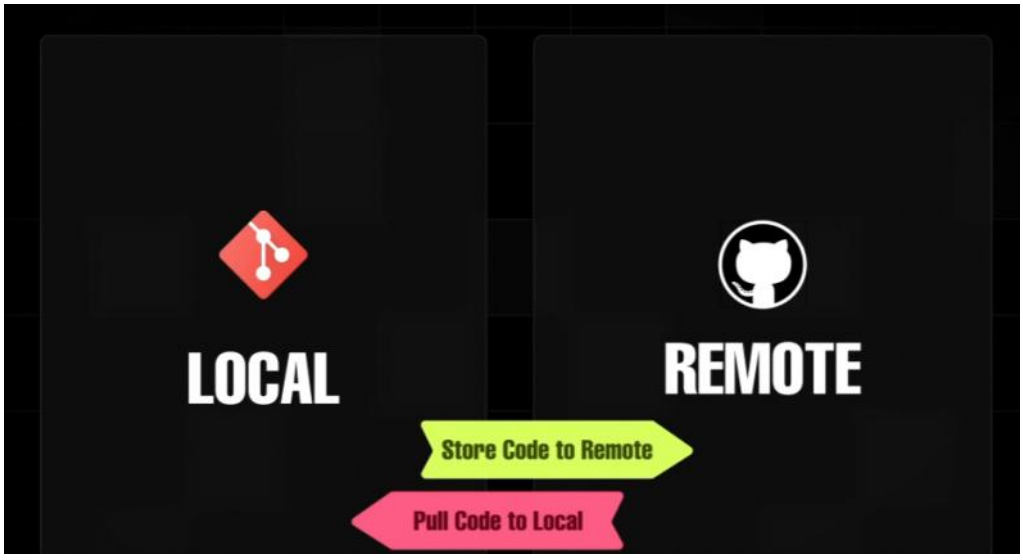
1
WORKING DIRECTORY
The folder where you're working on your project
2
STAGE
My changes are ready they can move to the next step
3
LOCAL REPOSITORY
Temporary area where files sit between working directory and repository save.
COMMIT
Local Git's Workflow

COMMIT
Committing means permanently saving those changes to your local repository
Local Git's Workflow

What is a Repository?
A Repository is a place where all the versions of your files and their complete change history are stored


Share Codes with Others
Access it from another computer
Keep it safely backed up
REMOTE


LOCAL
1
WORKING DIRECTORY
2
STAGE
3
LOCAL REPOSITORY
PUSH
REMOTE

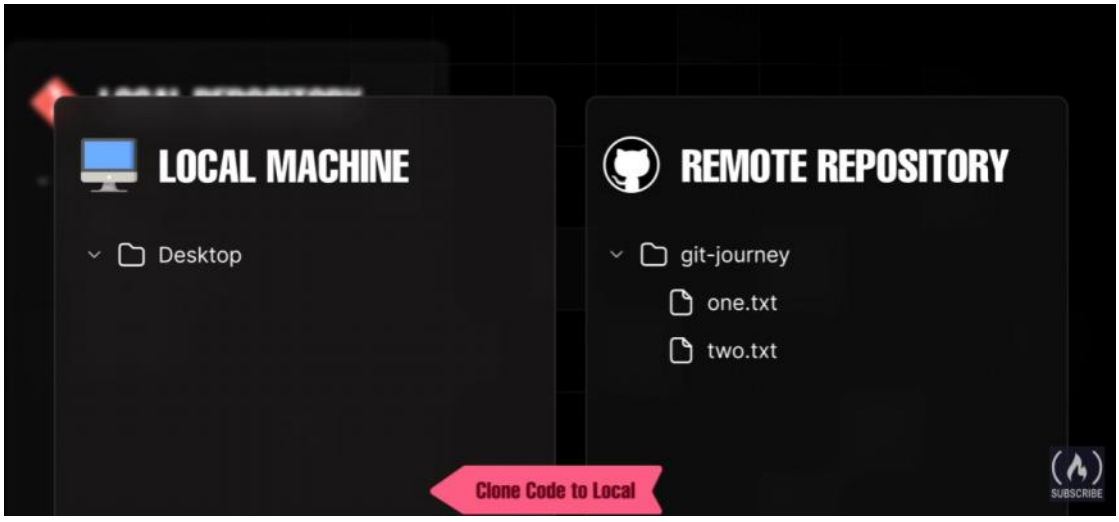

LOCAL
REMOTE
Store Code to Remote
Pull Code to Local

git-one
Recents
Shared
Favorites
Desktop
Applications
myFolder
one.txt
two.txt

```
➜ git-one git init
Initialized empty Git repository in /Users/rafe/Desktop/git-one/.git
/
➜ git-one git:(master) ✗ ls
myFolder one.txt  two.txt
➜ git-one git:(master) ✗ ls -la
total 32
drwxr-xr-x  7 rafe  staff   224 Nov 10 18:38 .
drwx------+ 5 rafe  staff   160 Nov 10 18:37 ..
-rw-r--r--@ 1 rafe  staff  6148 Nov 10 18:38 .DS_Store
drwxr-xr-x  9 rafe  staff   288 Nov 10 18:38 .git
drwxr-xr-x  3 rafe  staff    96 Nov 10 18:38 myFolder
-rw-r--r--@ 1 rafe  staff     3 Nov 10 18:38 one.txt
-rw-r--r--@ 1 rafe  staff     3 Nov 10 18:3  xt
➜ git-one git:(master) ✗
```

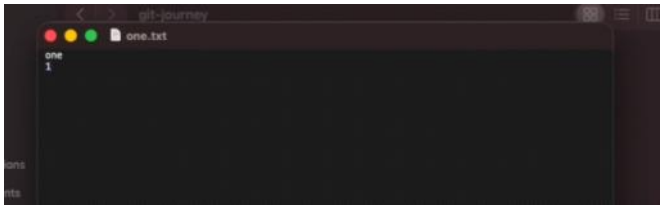
.git : core of the entire project

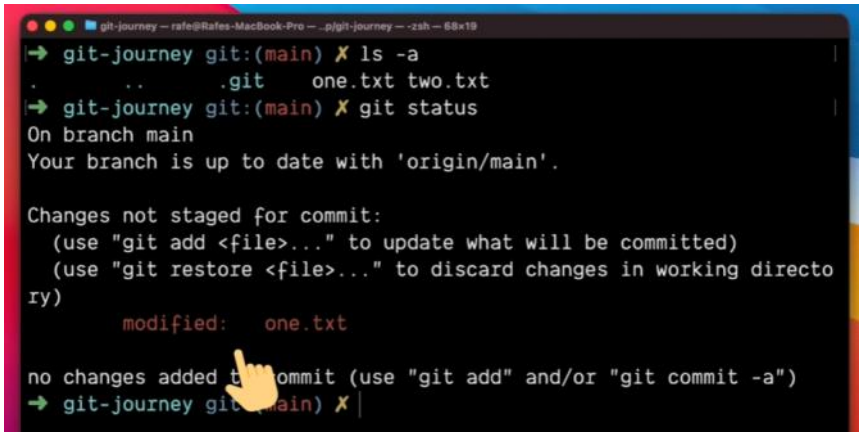
Git keeps all its internal data here such as file changes, who made the changes and versions

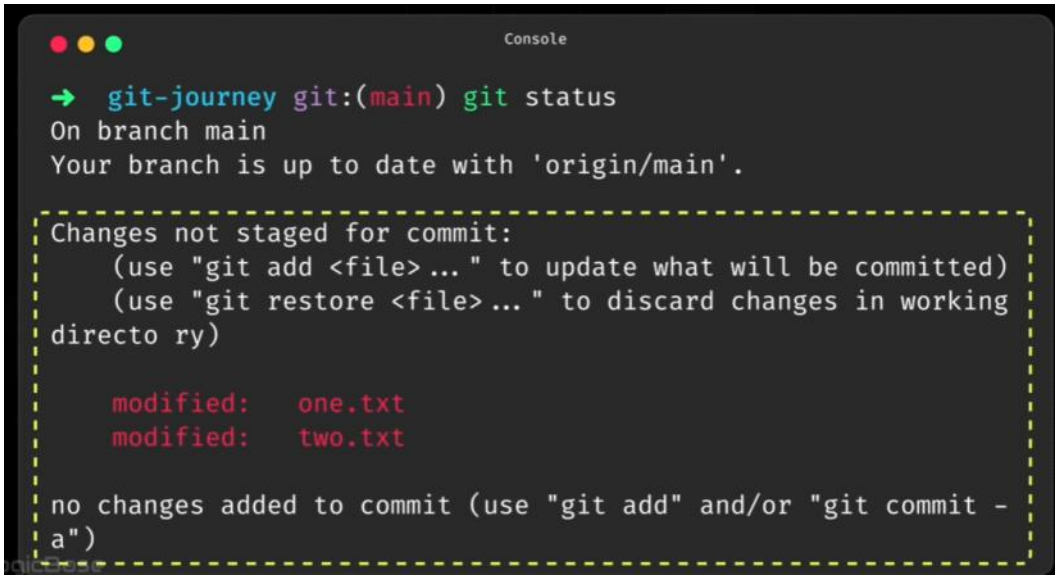



```
➜ Desktop git clone https://github.com/logicbaselabs/git-journey.gi
t
Cloning into 'git-journey'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0
)
Receiving objects: 100% (6/6), done.
➜ Desktop ls
git-journey git-one
```

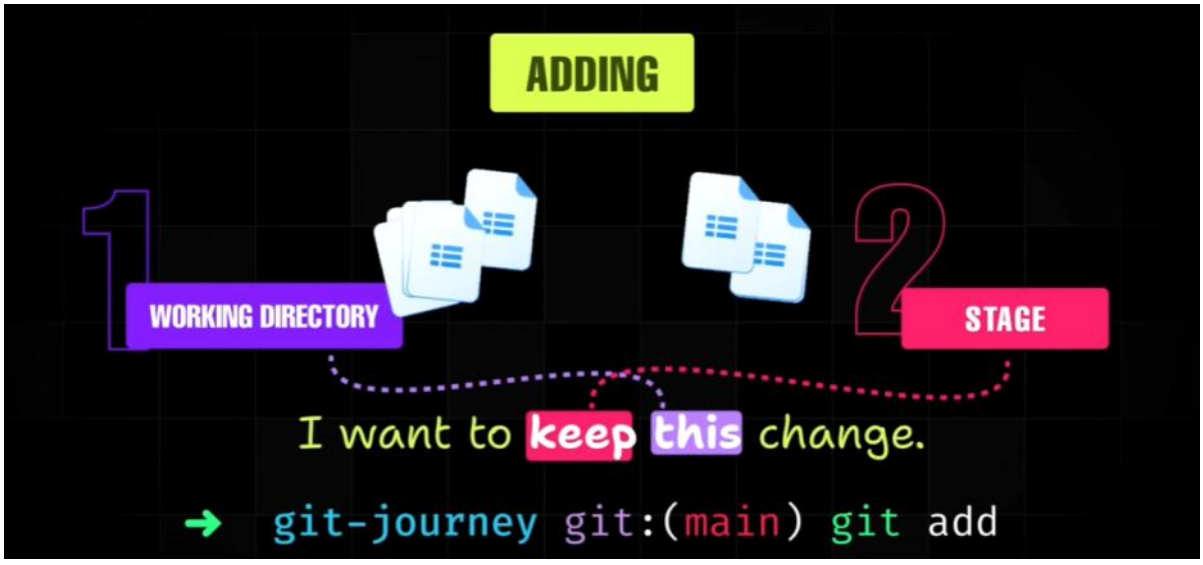
```
➜  git-journey ✗ git init
```

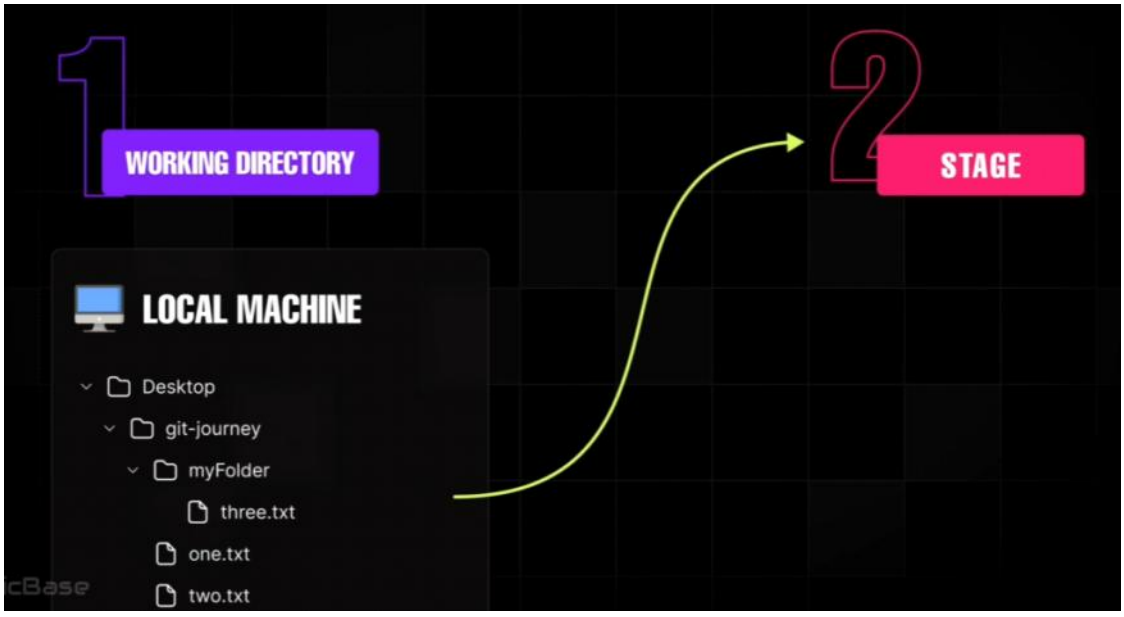

one.txt
one
1


➜ git-journey git:(main) ✗ ls -a
. .. .git one.txt two.txt
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directo
ry)
modified: one.txt
no changes added to commit (use "git add" and/or "git commit -a")
➜ git-journey git:(main) ✗


Console
➜ git-journey git:(main) git status
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working
directo ry)
modified: one.txt
modified: two.txt
no changes added to commit (use "git add" and/or "git commit -
a")

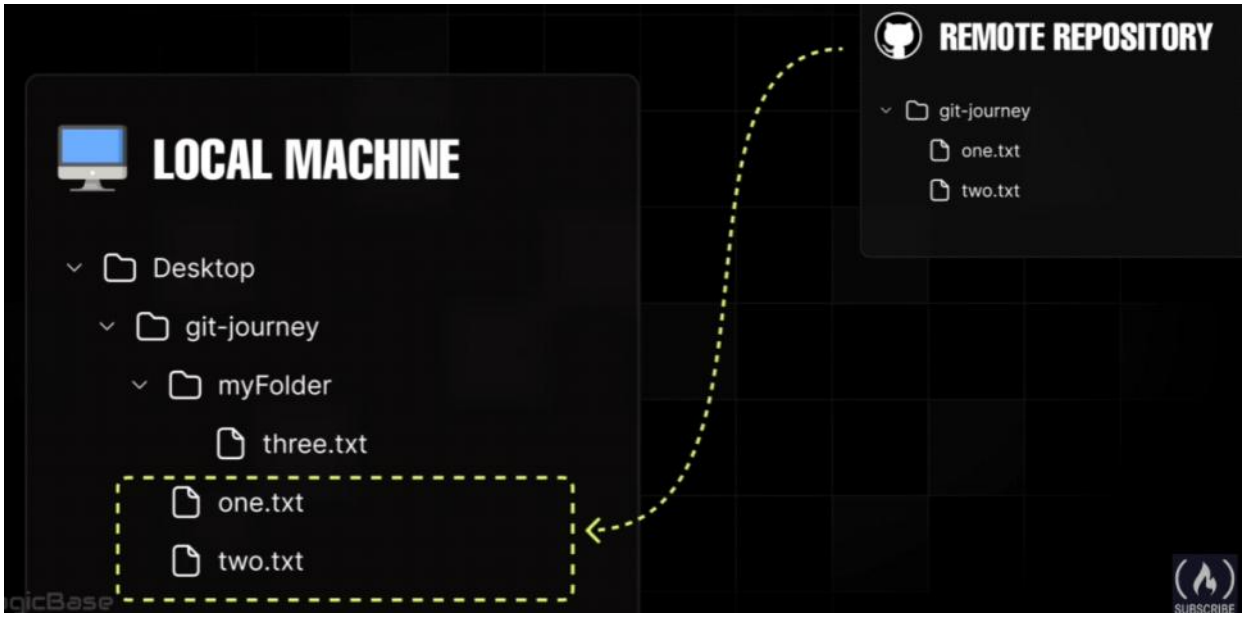
➜ git-journey git:(main) git status
What changes have been made in our working directory



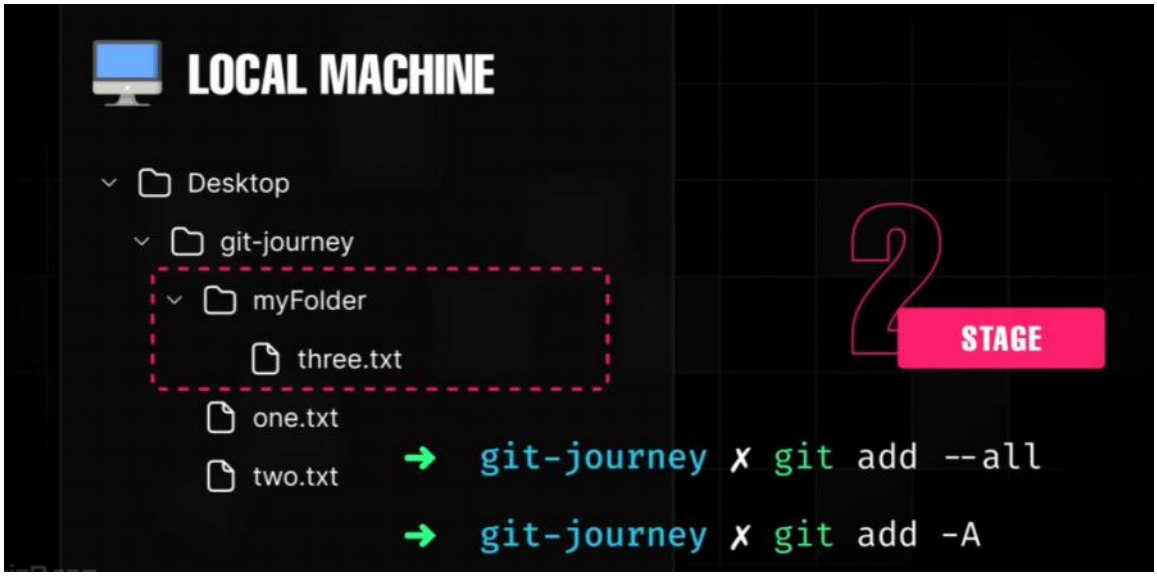



```
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directo
ry)
        modified:   one.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        myFolder/

no changes added to commit (use "git add" and/or "git commit -a")
➜ git-journey git:(main) ✗
```

myFolder/
is Untracked
As it is newly created and
Git has never been told to track it



Because the old files came from the remote repository so git already knows them, but since myFolder is newly created, it doesn't recognizes it yet




```
➜ git-journey git:(main) ✗ git add --all
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   myFolder/three.txt
        modified:   one.txt
        modified:   two.txt
```

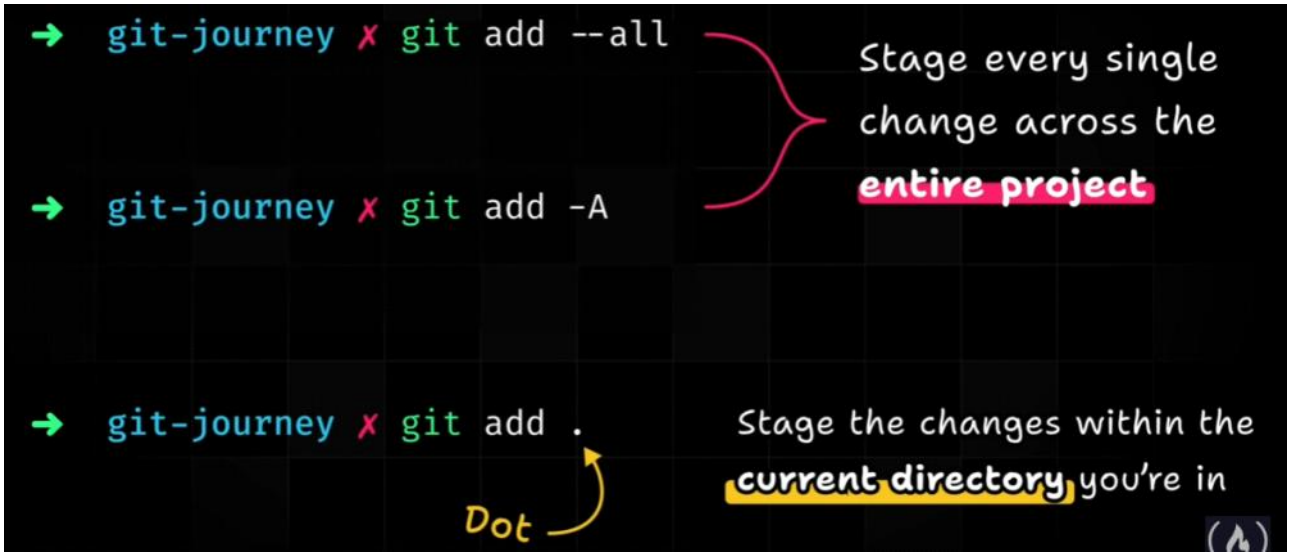
The staging area exists so you can choose **exactly what goes into the next commit**.

Without staging, every commit would automatically include all changed files.

```
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   myFolder/three.txt
        modified:   one.txt
        modified:   two.txt

➜ git-journey git:(main) ✗ git reset
Unstaged changes after reset:
M       one.txt
M       two.txt
```

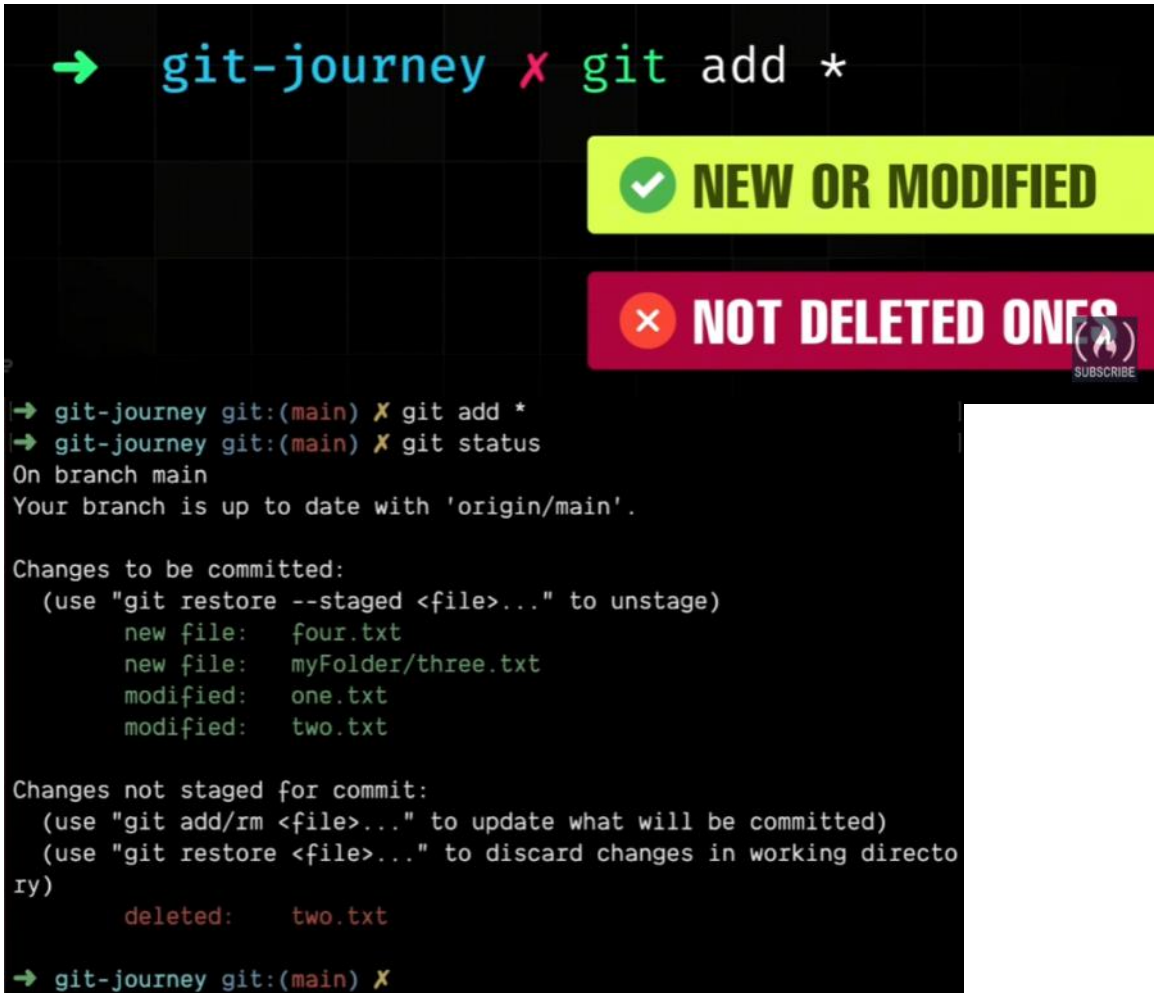
To go from Staging to previous state that is working directory
$ git reset

```
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directo
ry)
        modified:   one.txt
        modified:   two.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        myFolder/

no changes added to commit (use "git add" and/or "git commit -a")
```



```
ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   One.txt
        new file:   myFolder/three.txt
        modified:   two.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working direct
ory)
        deleted:    two.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        four.txt
```

After deleting "two.txt" and creating new file "four.txt"
And executing $ git status



Stages

```
➜  git-journey ✗ git add *
➜  git-journey ✗ git add --all
➜  git-journey ✗ git add -A
➜  git-journey ✗ git add .
```

Stages all visible changes except the deleted ones

--

Stages all

--

Stages just the current directory

-> git-journey$ reset
-> git-journey$ git add two.txt

What probably happened

Originally the file in Git/GitHub was:

```
One.txt
```

Later you renamed it in Windows to:

```
one.txt
```

But Windows treats both as "same file", so Git may not properly register the rename automatically.

Thus Git status still shows:

```
modified: One.txt
```

because Git tracks the old casing.

Verify actual Git-tracked name

Run:

```bash
git ls-files
```

You'll probably see:

```
One.txt
```

Proper way to rename case in Git

Git needs explicit rename:

```bash
git mv One.txt temp.txt
git mv temp.txt one.txt
```

```
ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git mv One.txt one.txt

ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   four.txt
        renamed:    One.txt -> one.txt
        deleted:    two.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   one.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        myFolder/
```

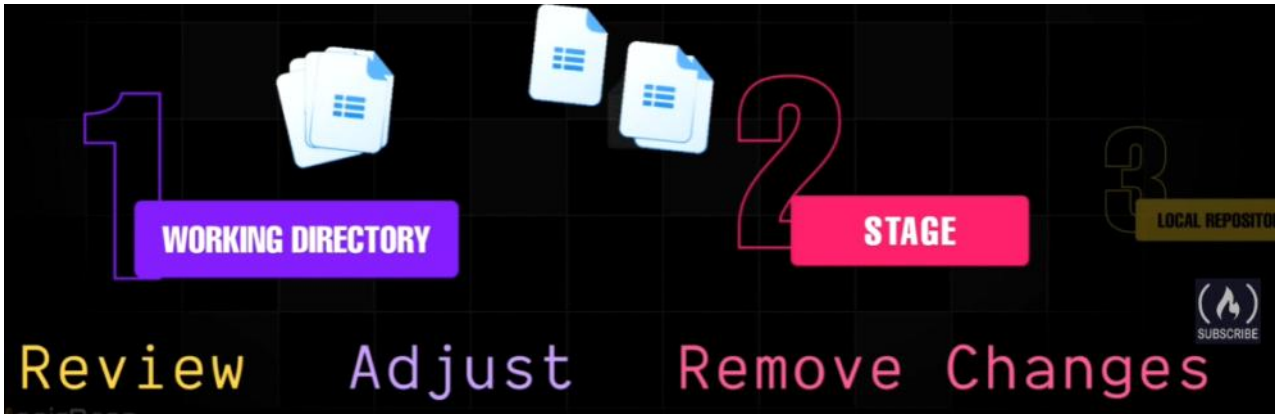
```
➜ git-journey git:(main) ✗ git add .
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   four.txt
        new file:   myFolder/three.txt
        modified:   one.txt
        deleted:    two.txt
```

Two stage everything, the best way is to

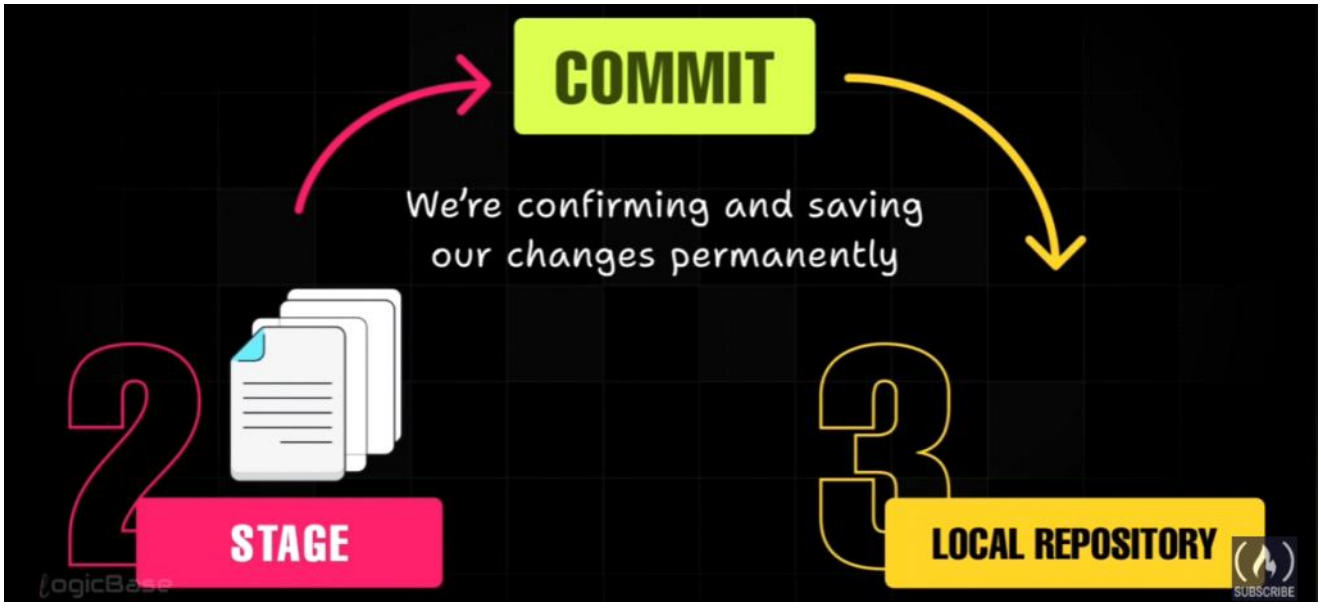
1) go to its root directory
2) and $ use git add .

Till now we learned:

3) Commit

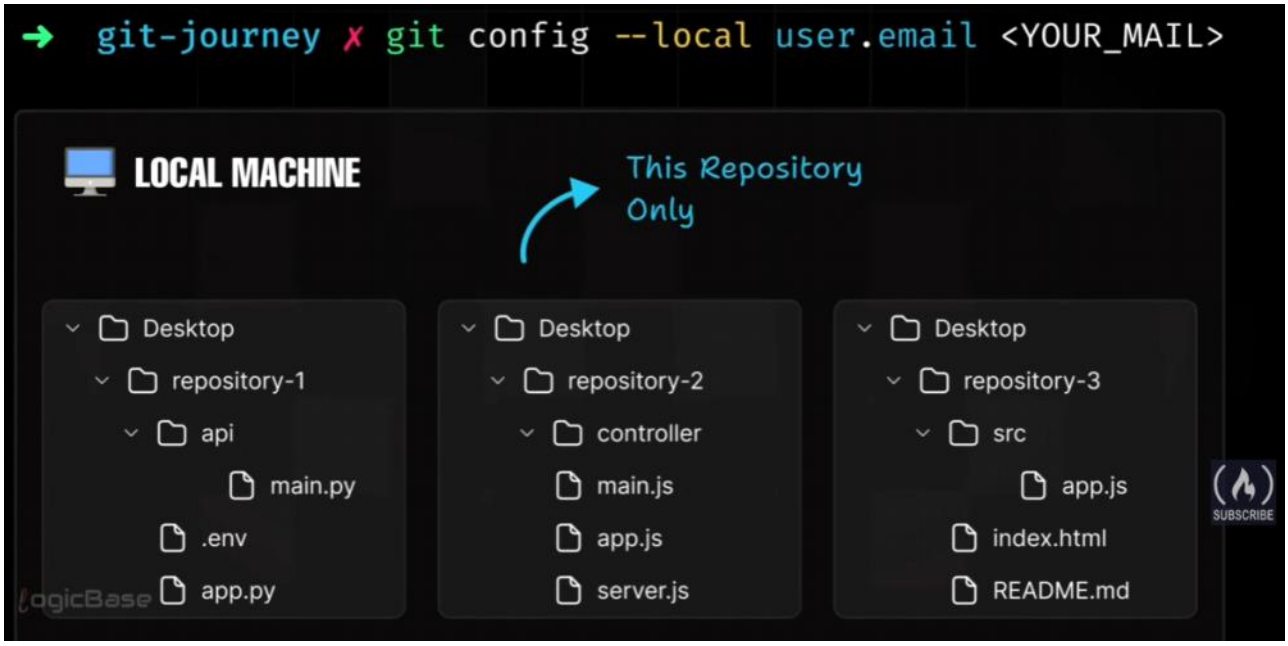


Before saving them permanently



```
➜ git-journey git:(main) ✗ git commit -m "I have made some changes
to files"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: no email was given and auto-detection is disabled
```

Making a commit



Undo the last Commit

```
➜ git-journey git:(main) git reset HEAD~
Unstaged changes after reset:
M       one.txt
D       two.txt
➜ git-journey git:(main) ✗
```

```
➜ git-journey git:(main) ✗ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directo
ry)
        modified:   one.txt
        deleted:    two.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        four.txt
        myFolder/

no changes added to commit (use "git add" and/or "git commit -a")
```

To delete and stage at the same time

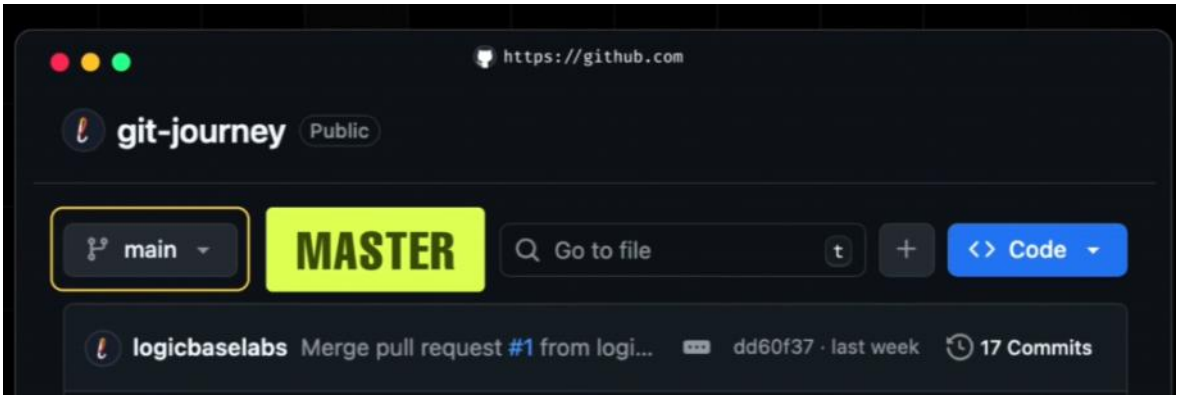
```
ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git rm four.txt
rm 'four.txt'

ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    four.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    one.txt
```

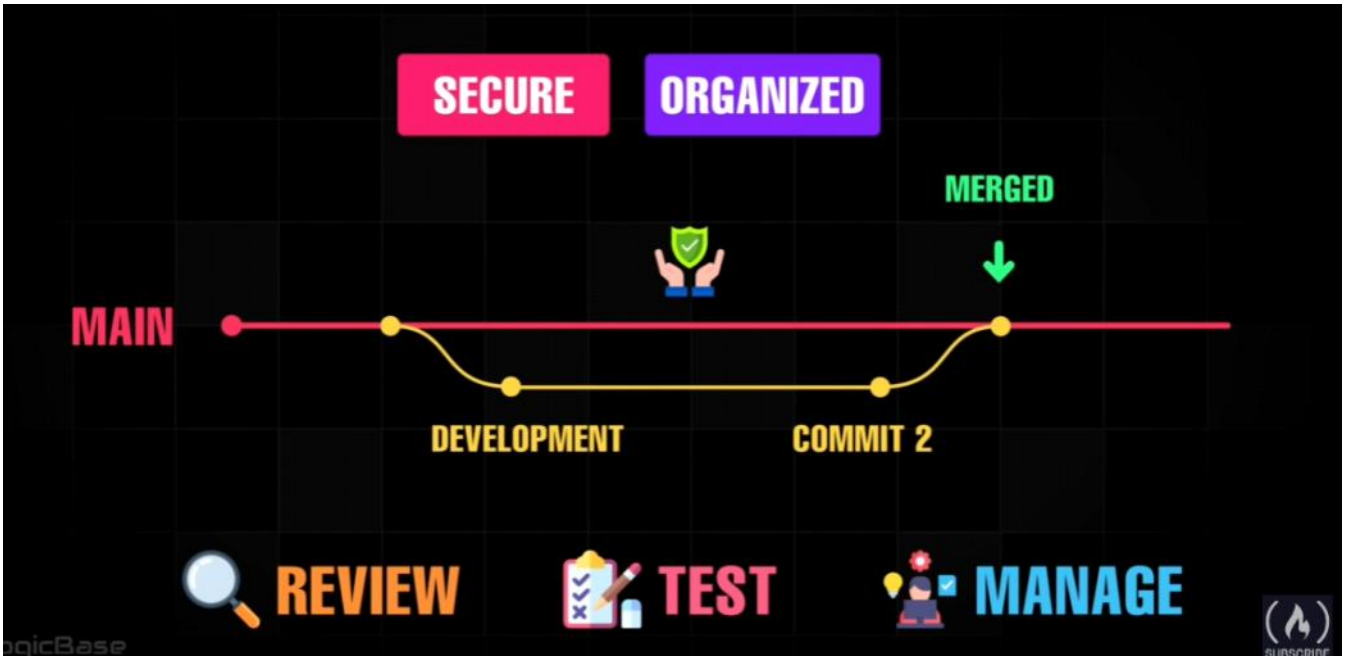
<u>Time stamp: 36:00 mins</u>

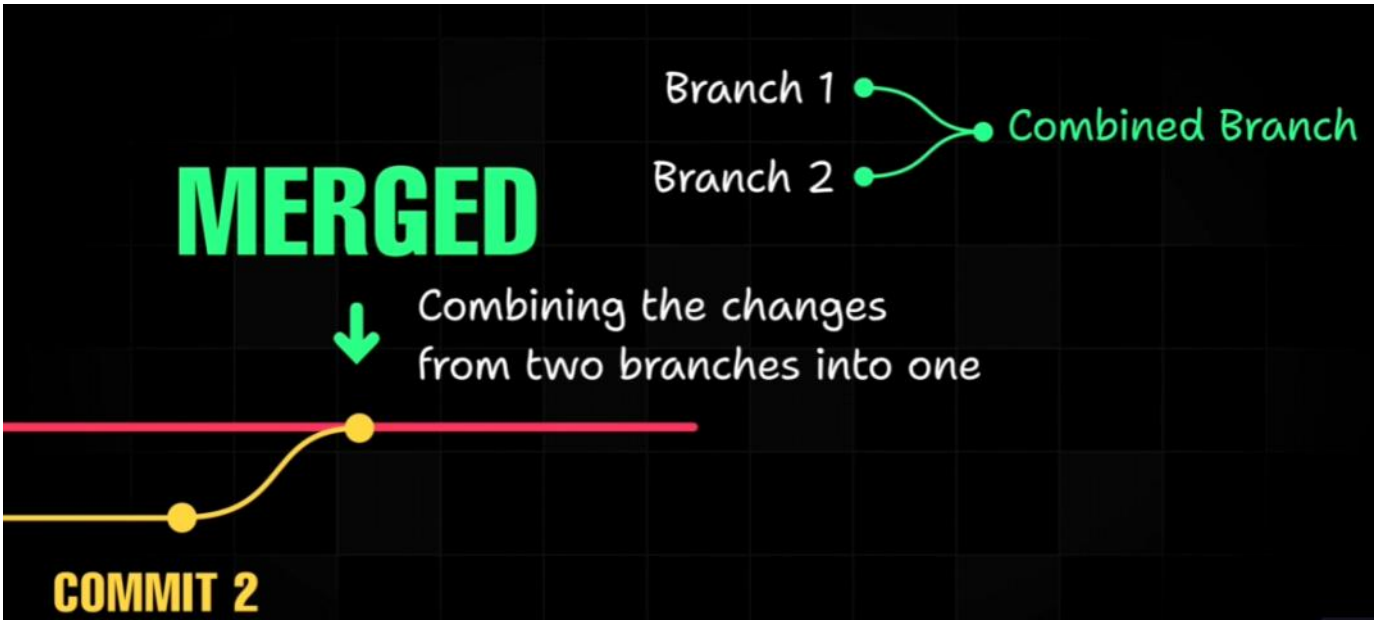
Git-Branching

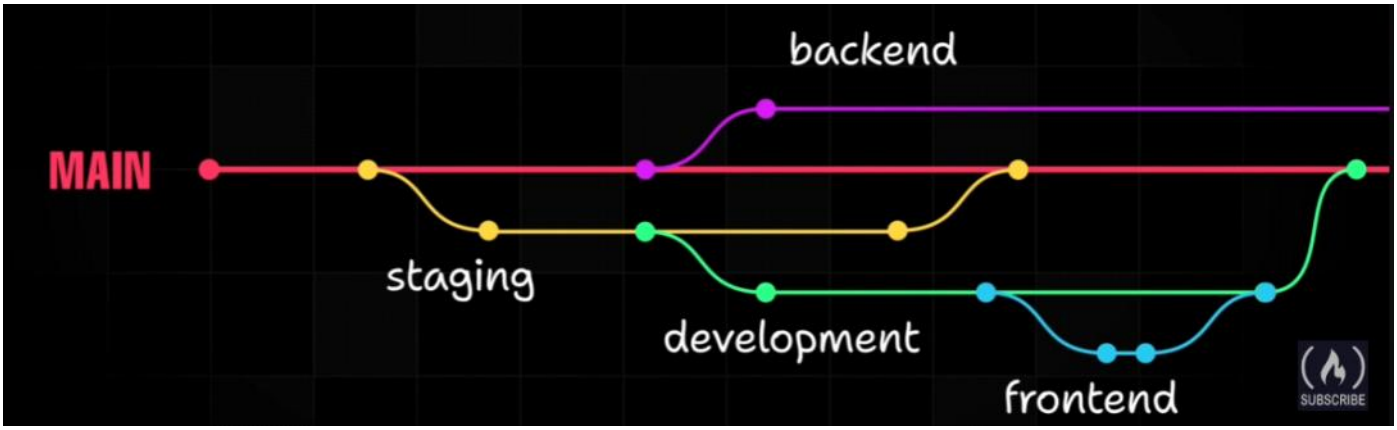
11 May 2026 06:41 PM

A Branch in Git is like a separate line of development where we can work independently









Check list of all Branches

```
ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git branch
* main
```

Create new Branch

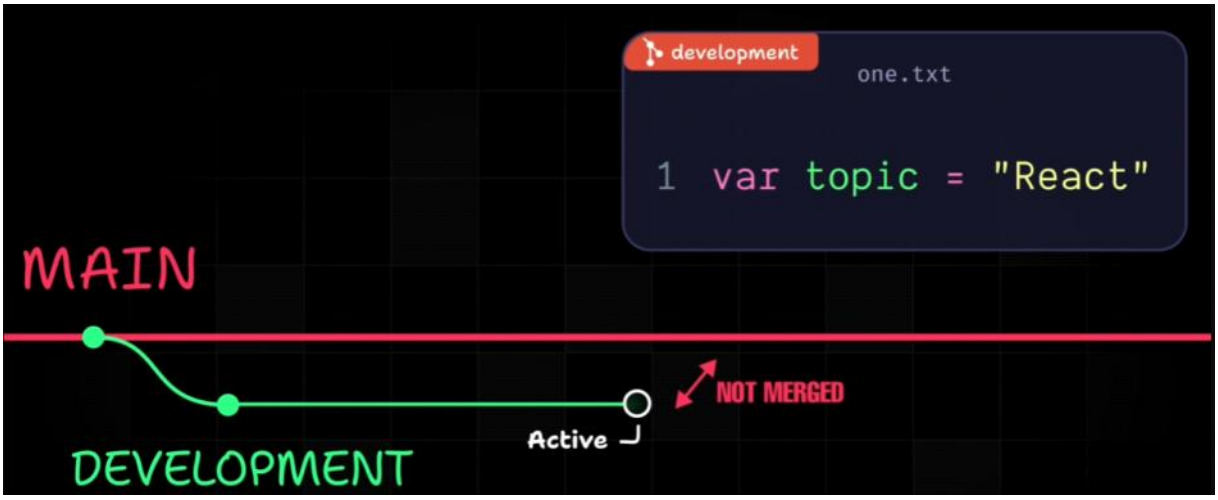
```
ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git branch development

ayush12@Ayush MINGW64 ~/Desktop/git-journey (main)
$ git branch
  development
* main
```

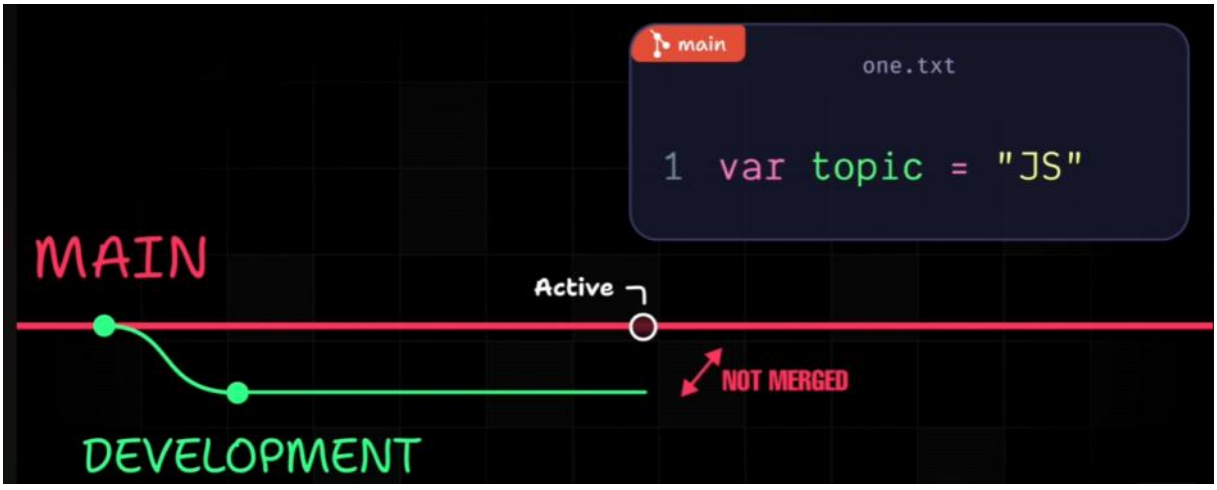
The star "" stands for the branch in which we are, here currently "main"*

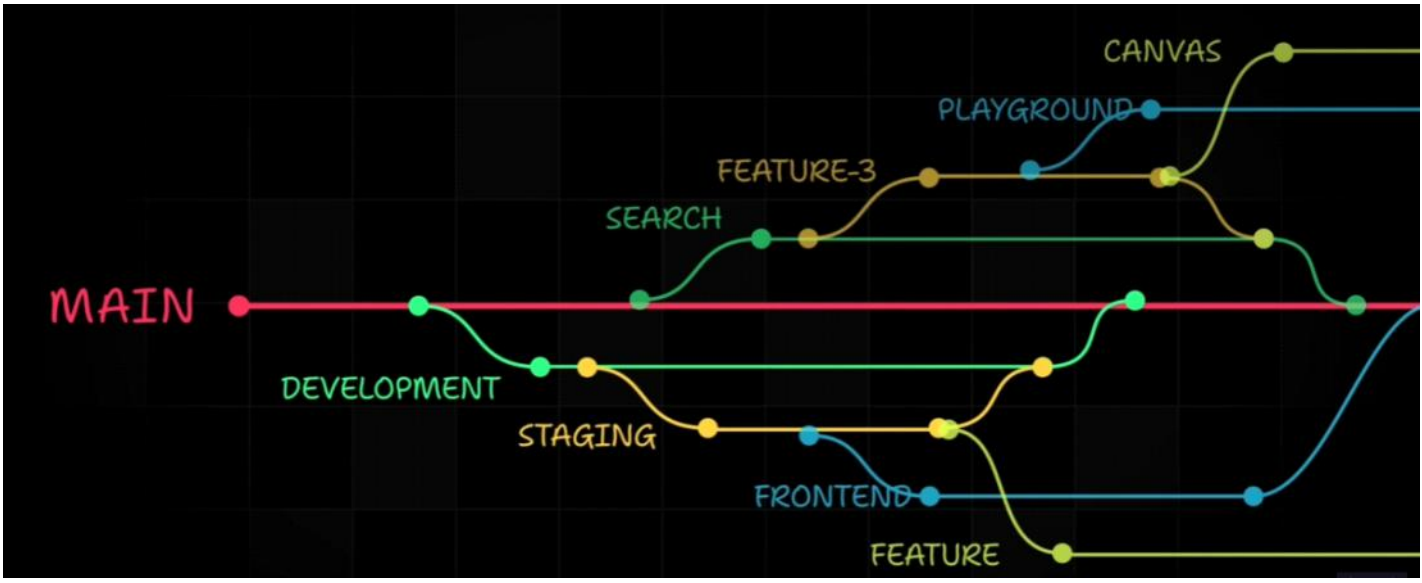
Switch to development branch

```
➜ git-journey git:(main) git branch
➜ git-journey git:(main) git branch development
➜ git-journey git:(main) git branch
➜ git-journey git:(main) git checkout development
Switched to branch 'development'
➜ git-journey git:(development)
```

Note: Changes made in development branch exists only in that branch
Since the branch is not merged, the moment we switched to main branch, git automatically hides the changes in the development branch showing only what exists in that main branch







Must me committed in each branch

Merging

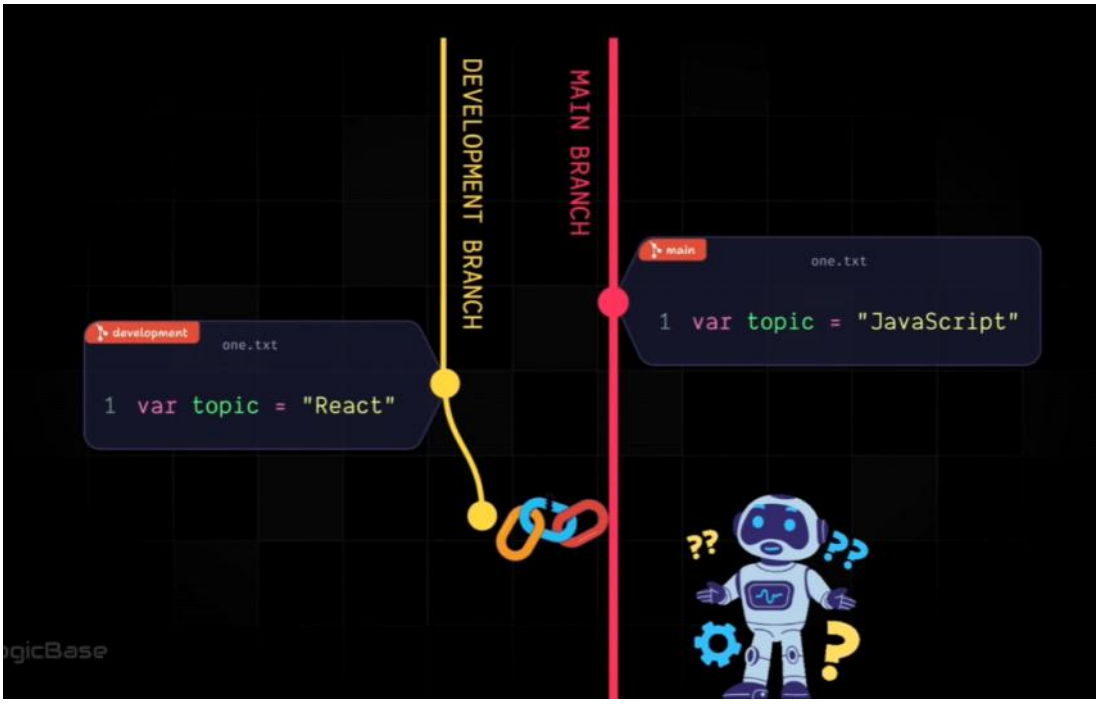
```
➜ git-journey git:(main) git checkout development
Switched to branch 'development'
➜ git-journey git:(development) git merge main -m "Merging main int
o development"
Merge made by the 'ort' strategy.
 four.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
➜ git-journey git:(development)
```

The changes made in the main branch has now appeared in the development branch as well

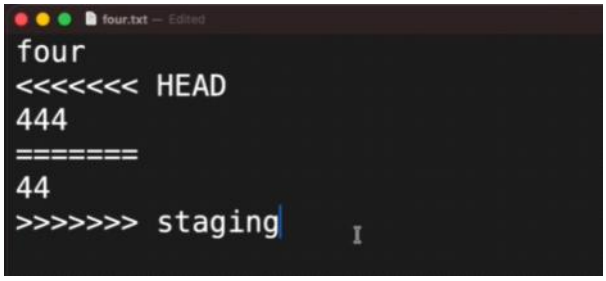
```
➜ git-journey git:(development) git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
➜ git-journey git:(main) git merge development -m "Merging on main
with development"
Updating 3030b28..9d8e918
Fast-forward (no commit created; -m option ignored)
 three.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 three.txt
```

The changes made in the development branch now appears in the main branch as a result of merging

Merge Conflict



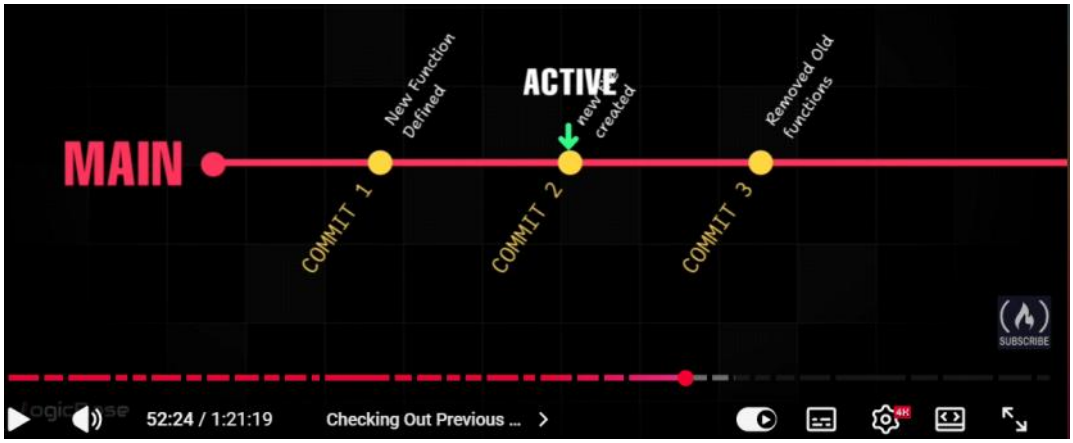
If an individual modified a file section in the main branch and someone else modified the same file section in development branch, git won't know which version to keep.
In such cases git flags the files as having conflict and need to resolve manually by deciding which one to keep



Here manually decide which section to keep
The main branch section or the staging branch section
Ex: say remove the staging section and all the conflicting markers and then add and commit the changes keeping the main branch section

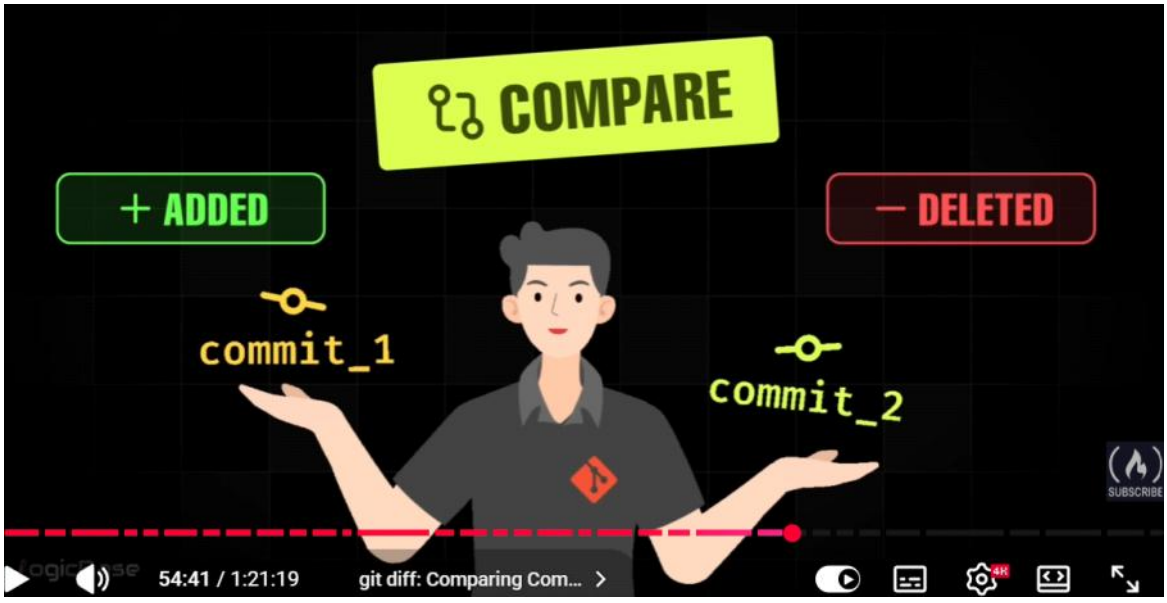
Checking Out the previous Commits

--Study--



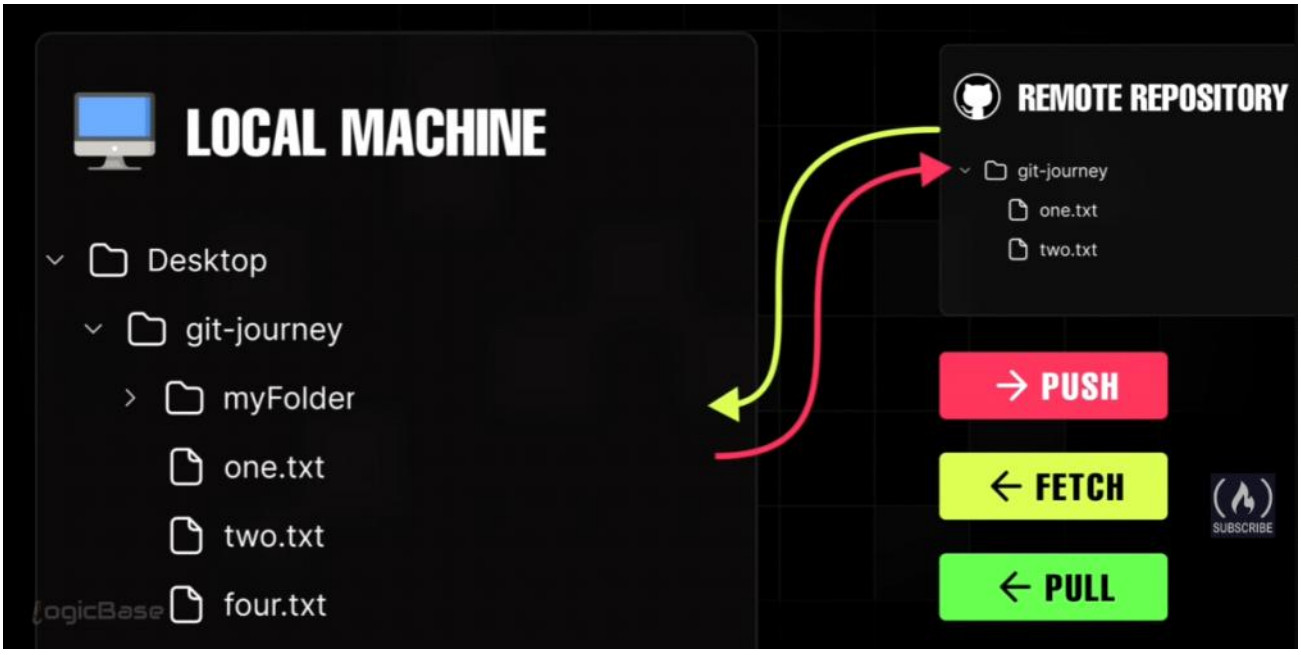
Comparing Commits

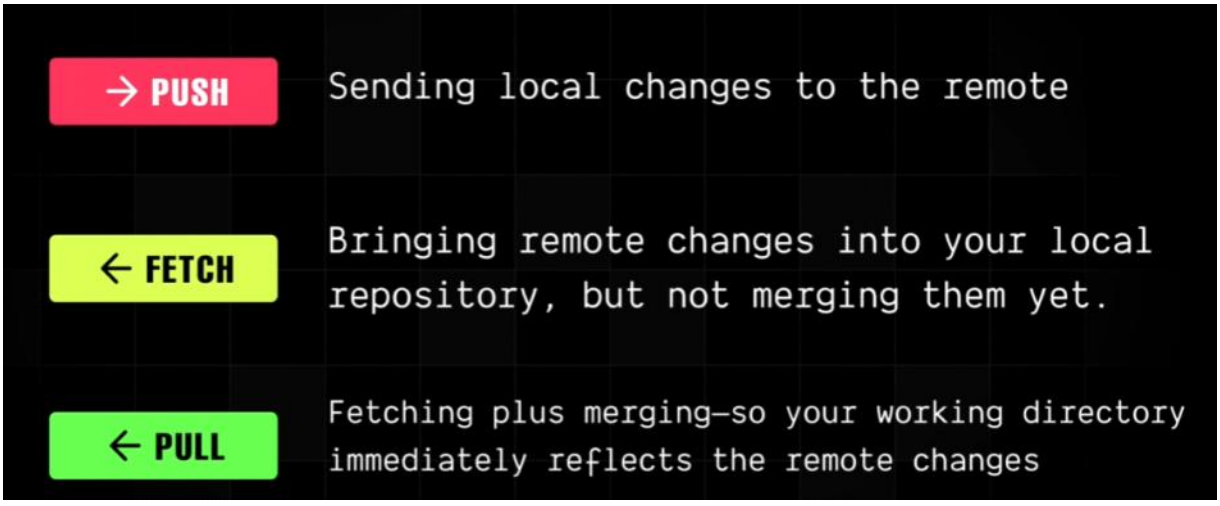
--Study--



Push, Fetch and Pull

11 May 2026 08:23 PM





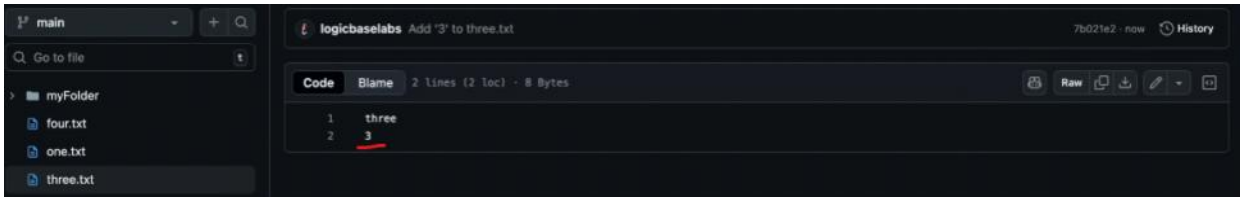
```
➜ git-journey git:(main) ✗ git push origin main
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
```

Origin: represents the remote repository and
Main is the branch we want to push to.

```
➜ git-journey git:(staging) ✗ git checkout development
Switched to branch 'development'
➜ git-journey git:(development) ✗ git push origin development
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'development' on GitHub by visitin
g:
remote:      https://github.com/logicbaselabs/git-journey/pull/new/d
evelopment
remote:
To github.com:logicbaselabs/git-journey.git
 * [new branch]      development -> development
```

Example

← FETCH



Say, made some changes in the remote repository

```
➜ git-journey git:(main) git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
```

Git fetches immediately the new changes from the remote repository
But those update don't show up in local file system until we merge the fetched changes

```
➜ git-journey git:(main) git merge
Updating f662069..7b021e2
Fast-forward
 three.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

Remote changes finally reflects in the local working directory as well

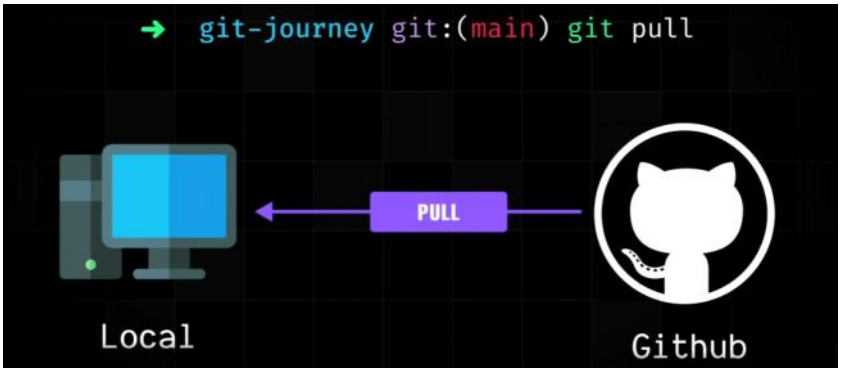
← PULL

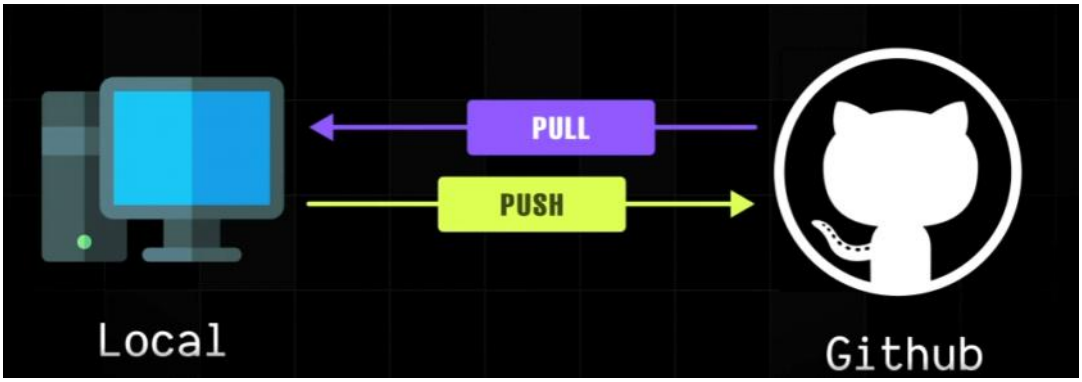
```
➜ git-journey git:(main) git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0
)
Unpacking objects: 100% (3/3), 925 bytes | 154.00 KiB/s, done.
From github.com:logicbaselabs/git-journey
   7b021e2..64a99df  main       -> origin/main
Updating 7b021e2..64a99df
Fast-forward
 three.txt | 1 +
 1 file changed, 1 insertion(+)
```

PULL = FETCH + MERGE

```
➜  git-journey git:(main) git pull
```

All the remote changes immediately appears in the local files



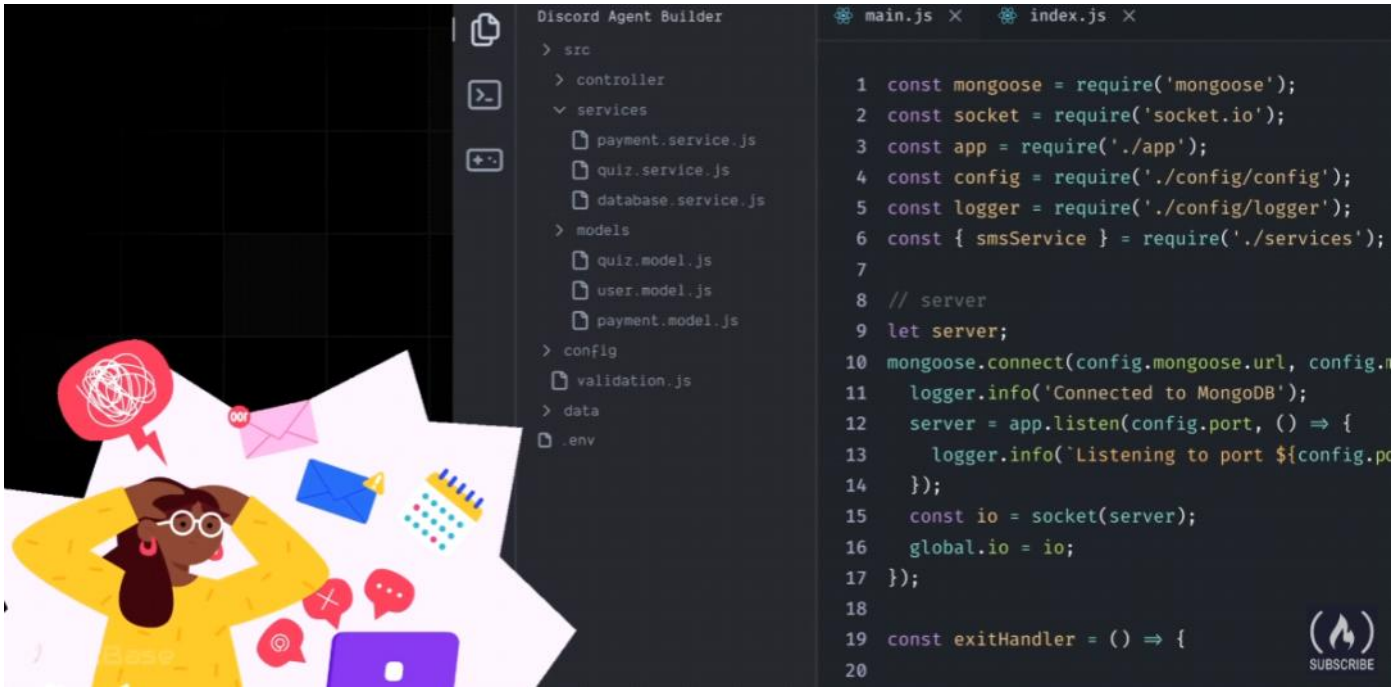



```
➜  git-journey git:(main) git push

➜  git-journey git:(main) git fetch

➜  git-journey git:(main) git pull
```

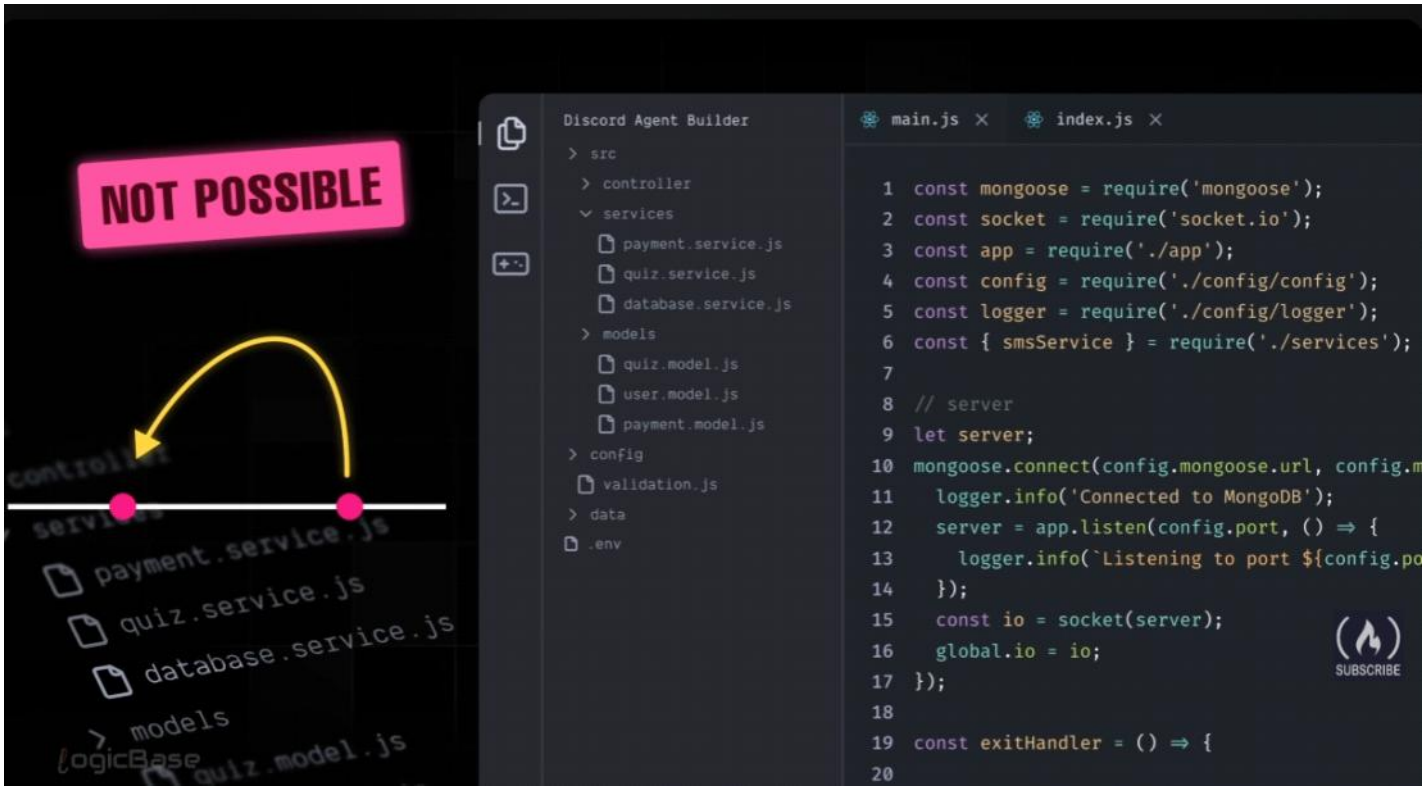
The above three are the most frequently used operations and are more than enough for most of our daily development work

Git Restore

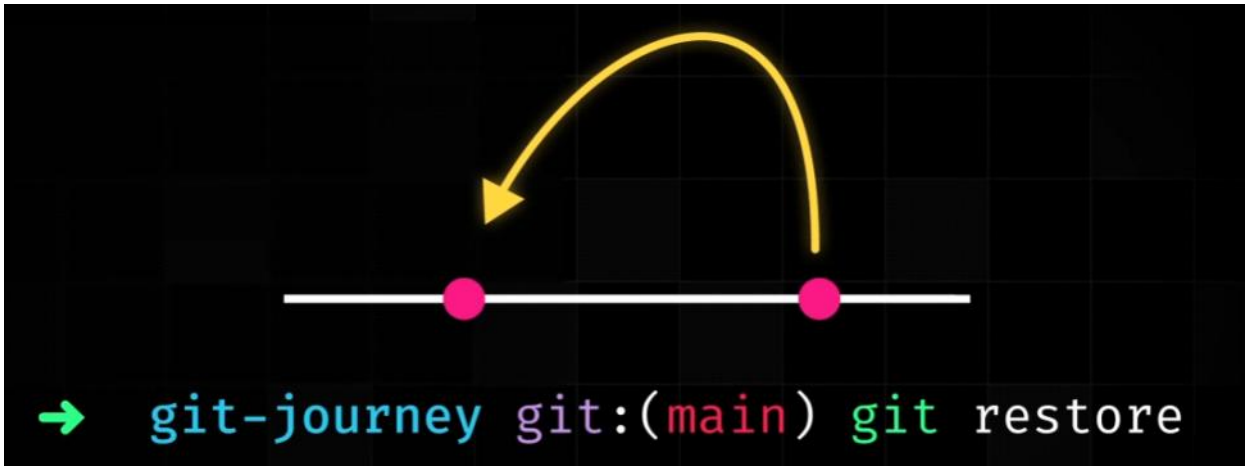
12 May 2026 04:05 PM



Wanted to go back to the previous committed state that is undo the newly changes made in or more than one file



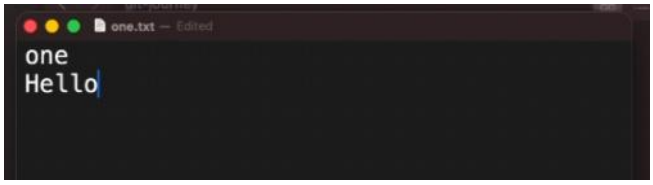
Visiting each file and removing the newly change is not always manually possible



git restore command helps you revert any files or directory to its previous state
It is mainly used for **uncommitted changes**.

Example:

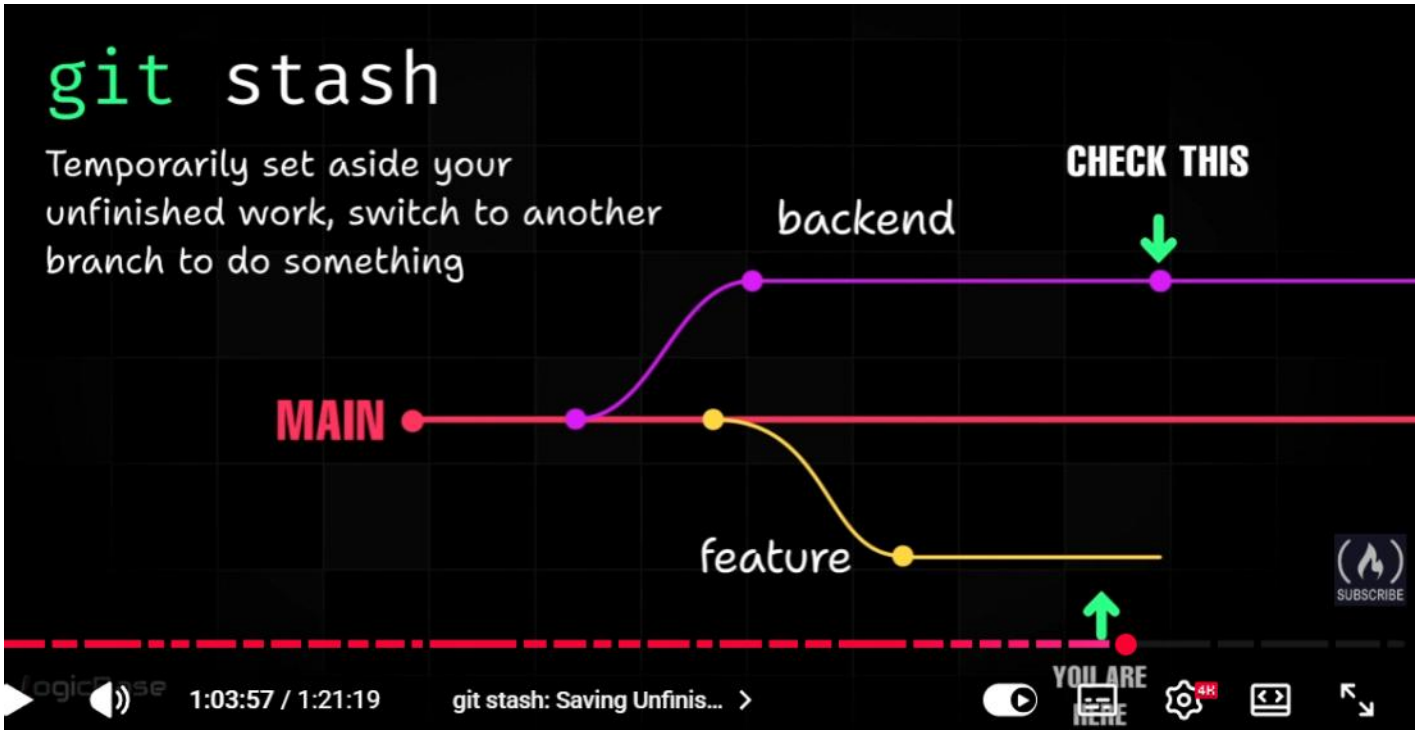
```
➜ git-journey git:(main) ✗ git restore one.txt
➜ git-journey git:(main)
```



Mainly used for undo local uncommitted changes

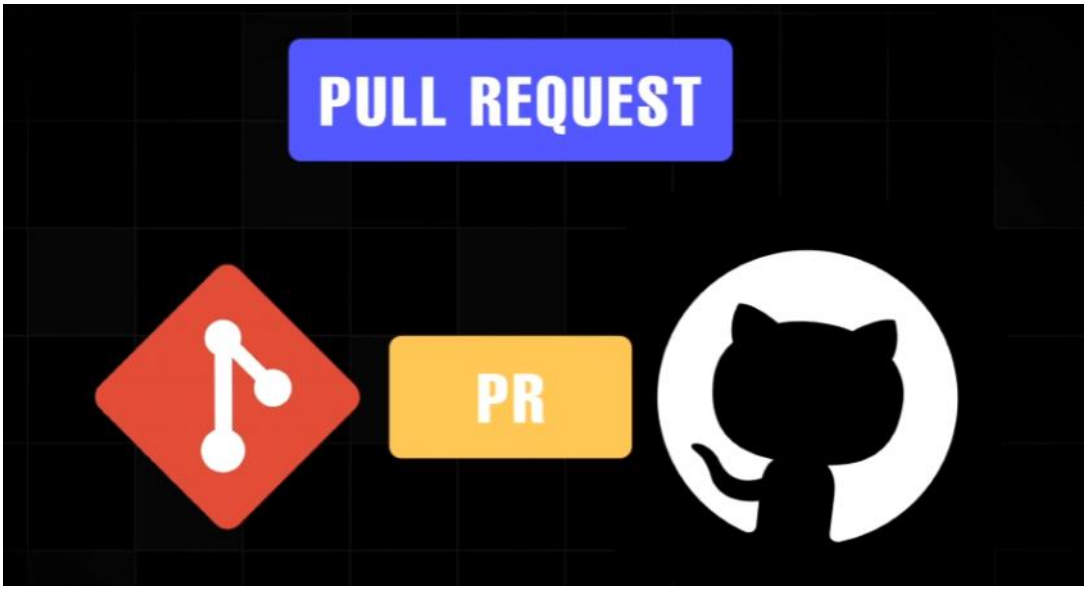
Git Stash

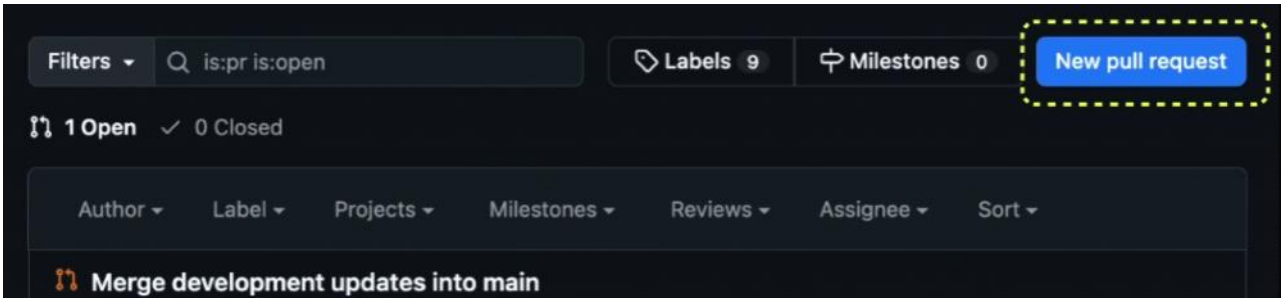
12 May 2026 04:20 PM



Pull Request

12 May 2026 04:26 PM





LogicBase Cheetsheet

12 May 2026 04:41 PM

Track & Save Changes

⚡Terminal⚡

```
# Show modified/unstaged files
git status

# Stage all changes
git add .

git commit -m "Describe what you changed"
```

Check for modified files, stage them for commit, and save with a descriptive message.

Connect to Remote Repo

⚡Terminal⚡

```
# Link to remote
git remote add origin <repo-url>

# Push code to main
git push -u origin main

# Pull latest from remote
git pull origin main
```

Link your local repo to a remote (like GitHub), push your commits, or fetch latest updates.

⏪ Revert or Reset Changes

⚡Terminal⚡

```
# Revert file to last commit
git restore <file>

# Unstage a file
git reset HEAD <file>

# Revert a commit with a new one
git revert <commit-id>
```

Undo local edits, unstage changes, or revert specific commits without breaking your repo.

🌿 Branching Made Easy

⚡Terminal⚡

```
# Create a new branch
git branch feature-x

# Switch to the new branch
git checkout feature-x

# Merge changes into current branch
git merge feature-x
```

Create a new line of development, switch between branches, and merge changes when ready.

⏪ Revert or Reset Changes

⚡Terminal⚡

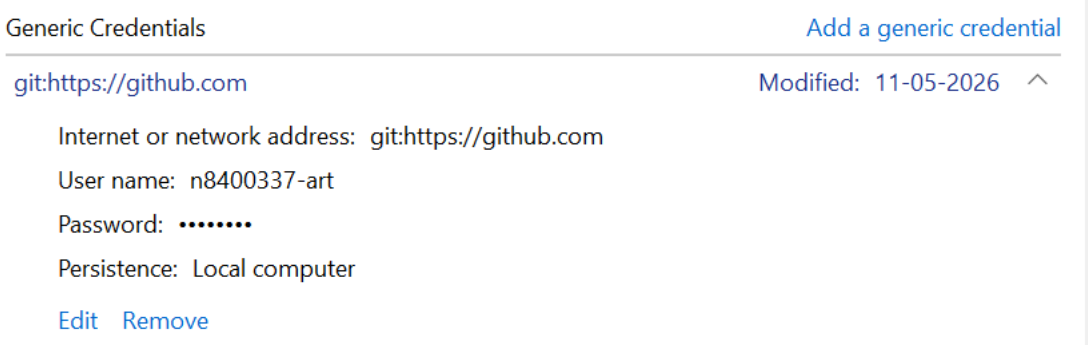
```
# Revert file to last commit
git restore <file>

# Unstage a file
git reset HEAD <file>

# Revert a commit with a new one
git revert <commit-id>
```

Undo local edits, unstage changes, or revert specific commits without breaking your repo.

Reset GIT Config

12 May 2026 05:06 PM

1. Got to windows icon and search for Credential Manager
2. Find Windows Credentials
3. Delete the git:https://github.com by clicking on Remove



4. Now go to Git Bash and Login as normal we do